Reporte de Avances: Plataforma HIL para el Control de un Péndulo Invertido
Diagnóstico y validación del modelo sin perturbación

Ing. Eduardo Pascual Pérez
Maestría en Robótica – UTM

1 de Noviembre del 2026

1. Introducción

Este reporte documenta el estado de la plataforma Hardware-in-the-Loop (HIL) del péndulo invertido en su configuración más simple: **sin perturbación y con todo el lazo ejecutándose en un solo núcleo (CPU1)** de la TMS320F28379D. El objetivo de esta etapa fue lograr que el sistema converja a 0 rad y que los datos que salen de la tarjeta sean confiables, de modo que sirvan como base para las etapas siguientes (perturbaciones y separación en dos núcleos).

El trabajo siguió una secuencia de diagnósticos encadenados, y el reporte está organizado en ese mismo orden:

1. **Enlace de datos y ruido** (Sección 3): se corrigió el baud rate, el ruido de la velocidad y la sincronización de la captura.
2. **Error en estado estacionario** (Sección 4): el ángulo no convergía a 0, sino a $\approx 0{,}014$ rad. Se localizó la causa en el lazo físico DAC→ADC, se descubrió que el reloj del sistema estaba a la mitad de la frecuencia prevista y se calibró el offset.
3. **Robustez de la adquisición** (Sección 5): se resolvieron un bloqueo de la telemetría y un artefacto en la primera muestra.
4. **Validación contra MATLAB** (Sección 6): se comparó el DSP con una simulación del mismo lazo. El régimen permanente coincide; queda una diferencia de amortiguamiento en el transitorio, todavía sin explicar.

Se indica en cada punto qué se midió y qué es todavía una hipótesis. La Sección 7 resume lo que no está verificado.

2. Arquitectura del sistema

2.1. Componentes

- **Firmware en el DSP** (`control_hil_v3.c`): dentro de la interrupción de Timer0 se simula la planta (péndulo invertido linealizado, integración de Euler-Cromer), se genera la salida por el DAC-B (pin 70), se lee el ADC-A0 (pin 30) a través de un jumper y se calcula el control PID. Ganancias: $K_P = 5$, $K_I = 2$, $K_D = 0{,}15$.
- **Middleware en Python**: recibe las tramas por UART (SCI-A), guarda un CSV y actualiza el gemelo digital en CoppeliaSim.

- **CoppeliaSim**: gemelo digital cinemático que muestra el estado del péndulo.
- **MATLAB**: post-procesamiento y simulación del lazo cerrado para comparar.

La planta es simulada, pero el ángulo *pasa por hardware real*: sale como voltaje por el DAC, regresa por el jumper y se digitaliza en el ADC. Eso es lo que hace al sistema HIL, y, como se verá, también es la fuente de las imperfecciones que se diagnosticaron.

El ángulo medido se reconstruye a partir del código del ADC:

$$\theta_{medido} = \theta_{min} + \frac{ADC - B}{M} \cdot \text{LSB}, \qquad \text{LSB} = \frac{\pi}{4095} = 0{,}7672 \text{ mrad}, \tag{1}$$

donde B y M son constantes de calibración (`ADC_CAL_B`, `ADC_CAL_M`; $M = 1$) que se introducen en la Sección 4.4.

2.2. Arquitectura multi-tasa

Conviven varias frecuencias de operación:

Tabla 1: Tasas de operación del sistema HIL

Tarea	Frecuencia	Período	Ejecutor
Integración de planta + PID	100 kHz	10 μs	ISR del DSP
Cálculo de derivada filtrada	100 Hz	10 ms	ISR (cada 1000 ejecuciones)
Reporte UART	100 Hz	10 ms	Bucle principal del DSP
Registro y post-procesamiento	100 Hz	10 ms	Python + MATLAB

El PID se ejecuta a 100 kHz y solo *reporta* su estado a 100 Hz. Por eso, la simulación de MATLAB debe ejecutar 1000 pasos de física por cada muestra del CSV para recorrer el mismo tiempo físico.

2.3. Protocolo de telemetría

Cada 10 ms el DSP envía una línea de texto con tres enteros separados por comas: `pos_mrad, vel_mrad_s,u_milli` (posición en mrad, velocidad en mrad/s y control en milésimas). La "velocidad" enviada es la derivada *filtrada* que calcula el DSP, no la velocidad angular verdadera. Al ser enteros, la telemetría tiene una resolución de 1 mrad; para medir medias con más finura se usa un acumulador interno del firmware que se lee con la ventana *Watch* de CCS.

3. Diagnóstico del enlace de datos y del ruido

3.1. Baud rate

Los primeros datos mostraban excepciones `UnicodeDecodeError` con bytes como `0x87` y `0x8F`. El DSP solo envía caracteres ASCII (dígitos, coma, signo menos, `CR`, `LF`), así que esos bytes solo pueden deberse a un desajuste de velocidad entre transmisor y receptor.

El generador de baud rate del SCI cumple

$$\text{Baud} = \frac{\text{LSPCLK}}{(\text{BRR} + 1) \cdot 8}. \tag{2}$$

El código suponía LSPCLK = 50 MHz, con lo que `SCILBAUD=53` daría $\approx$ 115,740 bps. Con un monitor de puerto serial independiente se comprobó que el enlace realmente operaba a **57,600 bps**:

a 115,200 se recibían bytes sin sentido y a 57,600 tramas válidas del tipo `"39,-8,224"`. Eso equivale a LSPCLK = 25 MHz:

$$\frac{25{,}000{,}000}{54 \cdot 8} \approx 57{,}870 \text{ bps}. \tag{3}$$

Como LSPCLK = SYSCLK/4 por defecto, esto indicaba que el SYSCLK estaba a 100 MHz y no a 200 MHz. La causa de ese reloj lento se encontró más adelante (Sección 4.3).

Se intentó confirmar el baud con un osciloscopio en GPIO42/GPIO43 (SCI-A), pero esos pines están ruteados al emulador XDS100v2 del LaunchPad para el puerto COM virtual y no están accesibles en los headers (SPRUII7), por lo que solo se midió ruido. La alternativa, migrar a SCI-B, requiere un adaptador USB-serial externo que no se tiene.

Corrección. Para 115,200 bps con LSPCLK = 25 MHz, BRR = 25,000,000/(115,200·8)−1 ≈ 26,1, es decir **BRR** = 26, que da 115,741 bps (+0,47 %). Algunas versiones de respaldo usaban 27, que con 25 MHz da 111,607 bps (−3,1 %). El firmware actual calcula

$$\text{BRR} = \frac{\text{LSPCLK} + 4\,\text{BAUD}}{8\,\text{BAUD}} - 1, \tag{4}$$

que da 26 con 25 MHz y 53 con 50 MHz (ambos +0,47 %), de modo que el enlace queda correcto con cualquiera de los dos relojes. Con `BAUD_RATE = 115200` en Python se obtuvo un CSV de 3342 filas en 33.4 s, es decir 100 Hz exactos.

3.2. Ruido en la velocidad: derivada numérica y filtro

Con el enlace corregido, la velocidad angular mostraba un ruido de ≈ ±5 rad/s, con picos de ±13 rad/s, excesivo para un péndulo que opera en ±1,57 rad. La causa era la derivada calculada a cada interrupción:

```
float theta_dot_meas = (theta_measured - theta_prev) / Ts;   // Ts = 10 us
```

Si el ruido de posición tiene desviación estándar σ_θ, el de la derivada numérica es

$$\sigma_{\dot\theta} = \frac{\sqrt{2}\,\sigma_\theta}{T_s}. \tag{5}$$

Con $\sigma_\theta = 1{,}61$ mrad (medición de octubre, ver nota más abajo) y $T_s = 10\,\mu$s resulta ≈ 228 rad/s, y con $T_s = 10$ ms resulta ≈ 0,228 rad/s: una reducción teórica de 1000 veces. Por eso la derivada pasó a calcularse cada 1000 interrupciones (10 ms), manteniéndose dentro del DSP para conservar la naturaleza HIL.

El filtro IIR original ($\alpha = 0{,}98$) tenía, a $T_s = 10\,\mu$s, una frecuencia de corte de ≈ 325 Hz, que no filtra nada útil para un péndulo con dinámica alrededor de 1 Hz. Al calcular la derivada a 10 ms, el coeficiente se recalculó para $f_c = 10$ Hz:

$$\alpha = \frac{2\pi f_c T_s}{1 + 2\pi f_c T_s} = \frac{2\pi \cdot 10 \cdot 0{,}01}{1 + 2\pi \cdot 10 \cdot 0{,}01} \approx 0{,}386. \tag{6}$$

```
contador_derivada++;
if (contador_derivada >= 1000) {          // cada 10 ms
    theta_dot_meas = (theta_measured - theta_prev_slow) / 0.01f;
    theta_prev_slow = theta_measured;
    contador_derivada = 0;
    theta_dot_filt = 0.614f * theta_dot_filt + 0.386f * theta_dot_meas;
}
```

Tabla 2: Efecto del cambio de derivada (observado en las gráficas)

Variable	**Antes** ($T_s = 10\,\mu s$)	**Después** ($T_s = 10\,ms$)
Velocidad (rad/s)	± 5 (picos ± 13)	$\pm 0{,}2$
Control (u)	± 2	$\pm 0{,}03$

La reducción observada (unas 25 veces en la velocidad) es menor que la teórica porque el filtro y otros efectos intervienen; el ruido restante ya no domina la señal.

Nota sobre σ_θ. El valor de 1.61 mrad proviene de una prueba de octubre con el DAC fijo en 2048 y el ADC en lazo, con 6399 muestras y pico a pico de 16 mrad. No se ha reconciliado con las mediciones posteriores: su media ($-1{,}5404$ rad) corresponde a un código de ADC cercano a 40, incompatible con un lazo con el DAC en 2048, y en el barrido de la Sección 4.2 la desviación por punto fue de 0.62 a 1.11 LSB (0.48 a 0.85 mrad). Por ello 1.61 mrad debe verse como una cota superior; la conclusión cualitativa (la derivada a T_s pequeño amplifica el ruido) no depende de ese valor.

3.3. Sincronización del inicio de la captura

El firmware original arrancaba el control al energizarse, mientras que Python tardaba $\approx 0{,}7$ s en abrir el puerto y empezar a grabar; el transitorio ocurría antes de que se registrara. Se implementó un *trigger* por UART: el firmware espera el byte `'S'`, reinicia el estado a la condición inicial, arranca el temporizador y responde `.ºK\r\n"`. Python envía la `'S'` justo antes de grabar, de modo que la primera muestra del CSV corresponde al inicio del control. El DSP atiende la `'S'` una sola vez por arranque: para repetir una corrida hay que reiniciar el programa en CCS.

4. Diagnóstico del error en estado estacionario

4.1. Síntoma

Con el ruido controlado, el ángulo no convergía a 0, sino que se estabilizaba en $\approx 0{,}014$ rad. Como el controlador y la planta son software, la sospecha recayó en lo único que no lo es: la cadena DAC→jumper→ADC.

4.2. Barrido DAC→ADC

Se escribió un firmware de diagnóstico que envía 65 códigos al DAC (0 a 4095 en pasos de 64) y mide media y desviación del ADC en cada punto. Con el sistema entonces a 100 MHz se obtuvo:

- Ajuste lineal: $\text{ADC} = 0{,}9906 \cdot \text{DAC} + 1{,}24$ LSB.
- Diferencia ADC$-$DAC: $+3{,}6$ LSB en el código 0, $-7{,}1$ en 1024, $\mathbf{-17{,}6}$ **en 2048**, $-29{,}8$ en 3072 y $-34{,}5$ en 4032.
- Residuos respecto de la recta de $-7{,}2$ a $+5{,}8$ LSB (escalones de 1 a 4 LSB: la cadena no es lineal al nivel de pocos LSB).
- Desviación por punto: 0.62 a 1.11 LSB.

Un offset de $-17{,}6$ LSB en la zona de trabajo equivale a $\approx 13{,}5$ mrad, que explica el error de $\approx 0{,}014$ rad. En estado estacionario, el integrador del PID lleva el *ángulo medido* a 0, así que el ángulo real queda

$$\theta_{real} = (B - \text{off}) \cdot \text{LSB}, \qquad \text{off} = \overline{\text{ADC} - \text{DAC}} \text{ en el punto de operación.} \tag{7}$$

Con $B = 0$ y off $= -17{,}6$ resulta $\theta_{real} = +13{,}5$ mrad, consistente con lo observado. (Un primer cálculo mío indicaba signo negativo y código de equilibrio ≈ 2030; al revisar la ecuación del firmware se corrigió a signo positivo y código ≈ 2065.)

4.3. El reloj del sistema estaba a 100 MHz en lugar de 200 MHz

Al agregar al firmware variables de diagnóstico calculadas de los registros de reloj (`sysclk_mhz_reg`, `lspclk_mhz_reg`) se comprobó lo que el baud rate ya sugería: SYSCLK = 100 MHz y LSPCLK = 25 MHz, con un tiempo máximo de ISR de hasta 1578 ciclos. La causa fue que el proyecto de CCS no tenía definido el símbolo predefinido `_LAUNCHXL_F28379D`. Un hilo de TI E2E indica que ese símbolo corrige el reloj lento en el LaunchPad; aquí se verificó de forma empírica. Un primer intento lo agregó en las propiedades *del archivo* y no tuvo efecto; al agregarlo a nivel *proyecto* (Properties → Build → C2000 Compiler → Predefined Symbols) y hacer *Clean*, los registros pasaron a SYSCLK = 200 MHz y LSPCLK = 50 MHz.

Consecuencias verificadas:

- La ISR cumple tiempo real: `isr_max_cycles` entre 1378 y 1578 de 2000 ciclos disponibles, `isr_overruns`= 0.
- Con el temporizador programado para 2000 ciclos, a 100 MHz el periodo de la ISR sería de 20 μs en lugar de 10 μs (deducción a partir del reloj, no medida directamente); con 200 MHz corresponde a los 100 kHz previstos. El periodo medio entre tramas medido a 200 MHz fue de 10.004 ms.
- El baud sigue correcto porque BRR se calcula según el reloj.
- **El offset del lazo DAC→ADC cambió**: de −17,6 LSB (100 MHz) a +6,6 LSB (200 MHz).

La causa de este cambio de offset **no está verificada**. La hipótesis es que depende de la ventana de adquisición o del reloj del ADC; no se asigna aquí una duración en nanosegundos a `ACQPS` porque las fuentes consultadas no coinciden en si cuenta ciclos de SYSCLK o del reloj del ADC. La consecuencia práctica es que *la calibración debe repetirse cada vez que cambie el reloj o la configuración del ADC.*

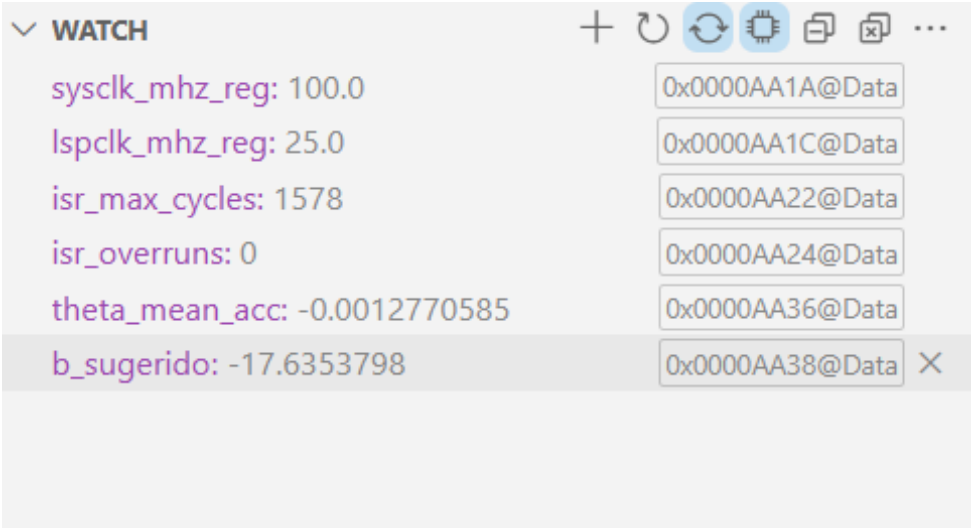


Figura 1: Ventana *Watch* con SYSCLK = 100 MHz: `b_sugerido` $\approx -17{,}6$ y `isr_max_cycles` = 1578.

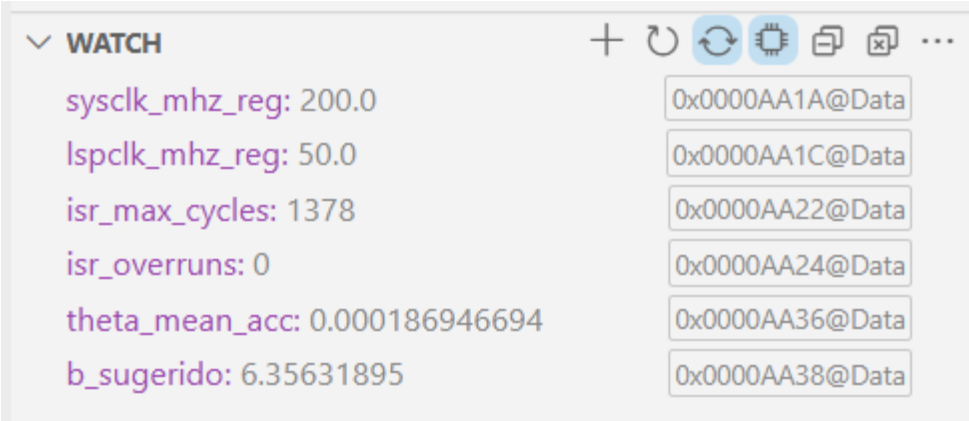


Figura 2: Ventana *Watch* con SYSCLK = 200 MHz: `b_sugerido` $\approx +6{,}6$.

4.4. Calibración del offset y resultado

El firmware mide el offset en línea: tras un asentamiento de 200 000 muestras, acumula 10^6 y calcula la media del ángulo (`theta_mean_acc`) y una estimación absoluta del offset, `b_sugerido` $= B -$ `theta_mean_acc`/LSB. Con el reloj a 200 MHz se fijó $B = 6{,}1$ LSB.

Tabla 3: Ángulo medio en régimen permanente (ventana *Watch*, 10^6 muestras)

Configuración	`theta_mean_acc` (mrad)
$B = 0$, 100 MHz (calculado del barrido, no medido con *Watch*)	$\approx +13{,}5$
$B = 6{,}6$, 200 MHz	0.187
$B = 6{,}1$, 200 MHz, repetición 1	0.537
$B = 6{,}1$, 200 MHz, repetición 2	0.440
$B = 6{,}1$, 200 MHz, repetición 3	0.459

El error estacionario pasó de ≈ 14 mrad a 0,2–0,5 mrad, es decir, menos de 1 LSB (0.767 mrad). **Esta calibración solo es válida para esta configuración de reloj/ADC y cerca del punto de equilibrio** (el barrido mostró escalones de 1 a 4 LSB en otros puntos). Falta la prueba de repetibilidad con tres reinicios y un apagado completo.

5. Robustez de la adquisición

5.1. Telemetría inundada de caracteres “9”

Con el trigger desactivado, la variable `tx_frames` permanecía en 0 y el monitor serial recibía un flujo interminable del carácter “9”. La causa era `sprintf` con `%ld`: en el C28x `int` es de 16 bits y la primera trama nunca terminaba. Se sustituyó por un formateador propio (`fmt_long`, que resta potencias de 10; comparado con `printf` para ± 999999) y una conversión saturada a milésimas (`to_milli`, NaN$\rightarrow$ 0). Después, el monitor mostró tramas válidas (6528 bytes en 6 s a 200 MHz) y `tx_frames` avanzó (1842 en una captura).

5.2. Middleware

El script de Python parecía congelado porque `readline()` de pyserial no regresa mientras lleguen bytes sin salto de línea (el flujo de “9”). La versión 2 incorpora: un lector de líneas con *timeout* real; hasta tres reintentos del trigger (acepta una línea de datos si se perdió el “OK”); filtro de líneas con exactamente tres enteros; escritura y vaciado del CSV *antes* de llamar a CoppeliaSim; un mensaje de estado cada 2 s; y un resumen final que compara el conteo de líneas

con el reloj real. El archivo se guarda con fecha y se copia a un nombre fijo que leen los scripts de MATLAB. En Windows las marcas de tiempo llegan en ráfagas de ≈ 16 ms (media 10.004 ms), por lo que el eje de tiempo se construye por conteo de líneas. Se probó con un puerto simulado (flujo normal e inundación) y en hardware (corrida de 20 s con 2000 líneas válidas).

Dos fallos de ejecución no estaban relacionados con el firmware: el error `object /pendulum_joint does not exist` (la pestaña activa de CoppeliaSim no era `Escena_Pendulo`) y una corrida sin la opción `-sin-coppelia` cuando solo se quería grabar el CSV.

5.3. Artefacto en la primera muestra

La rutina de reinicio inicializaba el DAC en 2048 (ángulo 0). La primera lectura del ADC daba entonces 0 rad aunque la condición inicial era $\theta_0 = 0{,}1$ rad, y la derivada interpretaba el salto como una velocidad inicial falsa de $\approx +2{,}96$ rad/s, que producía un pico inicial (hasta $\approx 0{,}15$ rad) que no es dinámica del péndulo. Ahora el DAC se inicializa con el código de θ_0 (2178) y se espera 200 μs antes de arrancar.

6. Validación contra una simulación en MATLAB

Para verificar que el DSP implementa el modelo previsto, se simula en MATLAB el mismo lazo: $\theta_0 = 0{,}1$ rad, $\dot{\theta}_0 = 0$, 1000 pasos de física de 10 μs por muestra, la derivada filtrada del DSP como "velocidad", eje de tiempo $t = (1:N) \cdot 0{,}01$ y el mismo redondeo a enteros de la telemetría. La primera versión del script tenía tres errores, ya corregidos: la condición inicial se tomaba de la columna de velocidad, se comparaba contra $\dot{\theta}$ en vez de la derivada filtrada y existía un desfase de 10 ms.

Tabla 4: Errores RMS DSP vs MATLAB (corrida posterior a la corrección del arranque)

Tramo	**Posición (rad)**	**Velocidad (rad/s)**	**Control (u)**
Total	0.00100	0.0474	0.0099
Transitorio ($\leq 0{,}6$ s)	0.00557	0.2204	0.0458
Permanente (≥ 2 s)	0.00024	0.0285	0.0061

En régimen permanente la media de la posición fue $+0{,}018$ mrad en el DSP y $-0{,}026$ mrad en MATLAB: coinciden dentro de la resolución de 1 mrad de la telemetría, lo que confirma la calibración. (Estas cifras provienen de la salida de MATLAB de esa corrida.)

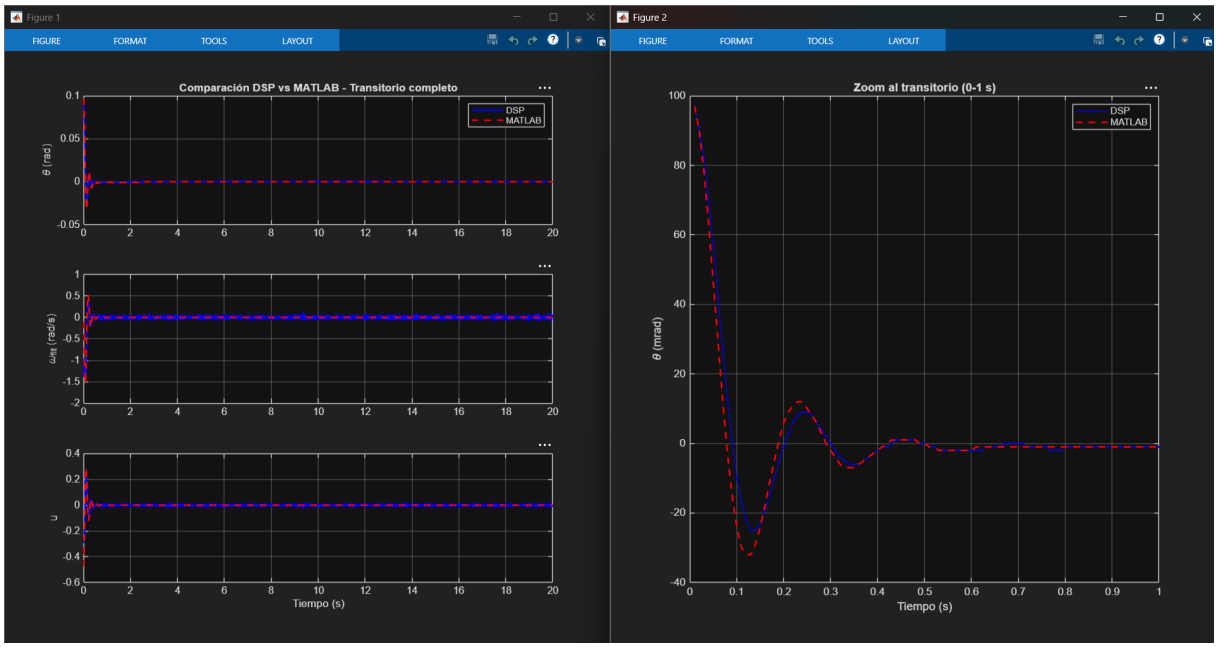


Figura 3: DSP vs MATLAB: transitorio completo (izquierda) y zoom de 0 a 1 s (derecha).

Diferencia abierta en el transitorio. El DSP está más amortiguado que MATLAB: el primer valle es de ≈ -25 mrad frente a ≈ -32 mrad. Con simulaciones auxiliares se descartó que un error de ganancia o un retardo en la medición lo expliquen; reducir K_P un 20 % o multiplicar la fricción por cuatro lo aproximan, pero **la causa no está confirmada**. Para investigarla se añadió al firmware la opción `DIAG_TELE_ERR`, que reemplaza la tercera columna por $\theta_{medido} - \theta$ (el error que introduce el lazo físico).

7. Alcance y límites de lo verificado

- **Verificado con datos:** el baud real del enlace; el offset del lazo DAC→ADC en dos configuraciones de reloj; que el reloj pasó de 100 a 200 MHz al definir el símbolo; que el error estacionario bajó de $\approx$ 14 a 0.2–0.5 mrad; que la ISR cumple tiempo real (sin *overruns*).
- **No verificado:** la causa del cambio de offset con el reloj; la repetibilidad de la calibración tras apagados completos; la causa de la diferencia de amortiguamiento; el valor del ruido del ADC (octubre vs. noviembre).
- **Alcance de la calibración:** válida para esta configuración y cerca del equilibrio.
- **Fuera de alcance de este reporte:** la respuesta con perturbaciones y la separación en dos núcleos.

8. Conclusiones

1. El enlace de datos es confiable: el baud real es de 115,741 bps (+0,47 %) con BRR calculado según el reloj, la telemetría llega a 100 Hz exactos y el middleware tolera pérdida de “OK”, basura y bloqueos.
2. El ruido de la velocidad se debía a derivar a $T_s = 10\,\mu$s; calcular la derivada cada 10 ms con un filtro IIR de 10 Hz lo redujo de ± 5 a $\pm 0{,}2$ rad/s sin sacar el cálculo del DSP.

3. El trigger por UART permite capturar el transitorio completo desde la primera muestra, y la corrección de la inicialización del DAC elimina un artefacto en el arranque.

4. El error estacionario de ≈ 14 mrad se debía al offset del lazo DAC→ADC, no al algoritmo. Se midió con un barrido y se compensó con `ADC_CAL_B`; el error quedó en 0.2–0.5 mrad (< 1 LSB).

5. Se descubrió que el reloj del sistema estaba a 100 MHz por falta del símbolo `_LAUNCHXL_F28379D`. Con 200 MHz la ISR cumple tiempo real y el offset del lazo cambia, por lo que la calibración debe repetirse si cambia la configuración.

6. Frente a una simulación en MATLAB, el DSP coincide en régimen permanente; en el transitorio persiste una diferencia de amortiguamiento cuya causa se está investigando.

9. Trabajo futuro inmediato

- Diagnosticar la diferencia de amortiguamiento del transitorio compilando con `DIAG_TELE_ERR`.
- Probar la repetibilidad de la calibración (tres reinicios y un apagado completo) y registrar `theta_mean_acc` y `b_sugerido`.
- Comparar `ACQPS`=14 con 99 para identificar la causa del cambio de offset con el reloj.
- Repetir el barrido DAC→ADC a 200 MHz (ajustando el baud del firmware de diagnóstico) y reconciliar la medición de ruido de octubre.
- Incorporar la perturbación y separar controlador y planta en los dos núcleos.